

コンパイラ演習 2026

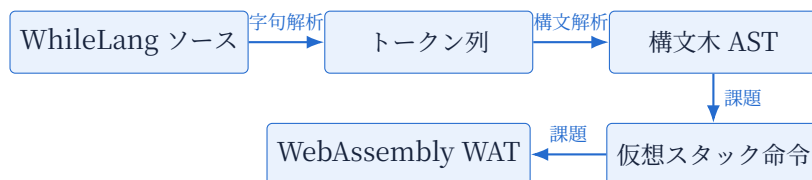
WhileLang から WebAssembly への翻訳

情報システム実験 I 集中講義用レジュメ

この演習では、小さな命令型言語 WhileLang を WebAssembly テキスト形式 `.wat` へ翻訳するコンパイラを実装します。字句解析器と構文解析器は配布済みであり、構文木から仮想スタック命令への翻訳と、その命令列から WAT への出力を完成させる。目標は、コンパイラの各段階を見通しながら、算術演算、比較演算、while 文の意味を保つ翻訳を実装することである。

1 コンパイラの見取り図

コンパイラは、ある言語で書いたプログラムを別の言語へ翻訳する。翻訳前後で表現は変わるが、プログラムを実行した結果は同じでなければならない。今回の処理は次の五段階で進む。



変更するファイルは次の二つだけである。

- Python: `virtual_stack.py` と `emit_wasm.py`
- C: `virtual_stack.c` と `emit_wasm.c`

最初に Python または C の一方を選んだら、途中で実装を混ぜない。

2 仮想スタック機械

仮想スタック命令は、値を積む、値を取り出す、二つの値を演算する、といった小さな操作からなる。二項演算では、左辺を翻訳し、右辺を翻訳し、最後に演算命令を置く。この並びは後置記法に相当する。

WhileLang

```
a := a * 2;
```

仮想スタック命令

```
rvalue a
push 2
*
lpush a
```

重要な区別

- `rvalue a` は変数 `a` の値を積む。
- `lpush a` はスタック先頭の値を変数 `a` に格納する。
- `*` では、下にある値を左辺、上にある値を右辺として計算する。

Wasm もスタック機械なので、仮想命令の多くは WAT の命令と一対一で対応する。

役割	仮想命令	WAT
定数を積む	push n	i32.const n
変数を読む	rvalue x	global.get \$x
変数へ書く	lpush x	global.set \$x
加算	+	i32.add
符号付き比較	<	i32.lt_s
出力	print	call \$print

3 課題と実装のヒント

3.1 課題 1 算術演算

実装内容は、Sub / Mul / Div の構文木を仮想命令へ翻訳することと、Minus / Times に対して i32.sub / i32.mul を出力することである。割り算の WAT 出力 i32.div_s は配布済みである。

```
case Add(lhs, rhs):
    return compile_arith(lhs) + compile_arith(rhs) + [Plus()]
```

Add の場合を写し、末尾の命令を変えればよい。C では ARITH_ADD と補助関数 bin_arith を手本にする。引き算 10 - 3 では 10、3 の順に積む。print 3 - 10; が -7 になれば順序は正しい。

3.2 課題 2 比較演算

実装内容は、GT / GE / LE / EQ を仮想命令へ翻訳し、対応する WAT を出力することである。LT が完成例になっている。

比較	WAT	比較	WAT
>	i32.gt_s	>=	i32.ge_s
<=	i32.le_s	==	i32.eq

比較結果は真なら 1、偽なら 0 になる。print i > 1; のように、比較式を直接出力して確認できる。

3.3 課題 3 while

while 文には、条件を調べる場所 test と、ループを抜けた後の場所 out が必要である。

```
while i < 10 do
  i := i + 1;
```

```
label    L.0      # test
rvalue i
push     10
<
gofalse L.1      # out
rvalue i
push     1
+
lpush    i
goto     L.0
label    L.1
```

命令を組み立てる順序は、LabelTest、条件、GoFalse、本体、GoTo、LabelOut である。GoFalse は条件が 0 のとき out へ進み、GoTo は無条件に test へ戻る。ラベル名は各 while ごとに `gen_label()` で新しく作れば、入れ子にも対応できる。

WAT では block が脱出先を、内側の loop が反復先を表す。ここを出力する処理は配布済みである。

```
(block $L.1
  (loop $L.0
    ... condition ...
    i32.eqz
    br_if $L.1
    ... body ...
    br $L.0
  )
)
```

よくある失敗

- GoFalse と GoTo を逆にして、一度も回らない、または無限ループになる。
- LabelTest(test, out) と LabelOut(test, out) に別の組を渡す。
- while の本体が複数文なのに、入力側で `begin ... end` を使わない。

4 デバッグと確認手順

不具合は、翻訳パイプラインの手前から順に調べる。AST が期待どおりならパーサは正しく、翻訳側を調べればよい。WAT を調べる前に `whilei` で仮想命令が正しく動くことを確認する。

1. 構文木を見る。

```
python3 whilec.py --ast ../test/arith.while
```

2. 仮想命令列を見る。

```
python3 whilec.py --stack ../test/arith.while
```

3. 一命令ずつスタックと変数を追う。

```
python3 whilei.py --trace ../test/arith.while
```

4. 全テストを実行する。

```
python3 test_whilelang.py
```

C 版では `./whilec`、`./whilei` に読み替え、全テストは `make test` で実行する。Windows では Python を `py -3` で起動できる。

入力	期待する出力	確認する内容
<code>assign.while</code>	3	配布状態と加算
<code>arith.while</code>	7 8 4 23	減算、乗算、除算、優先順位
<code>cmp.while</code>	1 1 0 1 1 0	比較結果
<code>simple_loop.while</code>	10	基本の while
<code>fact.while</code>	120	while と乗算
<code>loop.while</code>	入れ子の出力	ラベルの組

5 着手順と発展課題

最初に配布状態で `assign.while` が 3 を出すことを確かめる。その後、課題 1、課題 2、課題 3 の順に進み、短いテストを通してから次へ移る。詰まったときはリポジトリ直下の `HINTS.md` を上から読む。

時間が余ったら、`if P then S else S` を実装する。命令の順序は、条件、`IfStart`、`then` 節、`ElseOp`、`else` 節、`IfEnd` である。Python の命令名は `IfStart` / `ElseOp` / `IfEnd`、C では `I_IF_START` / `I_ELSE` / `I_IF_END` である。